



Figure 9: Kinds of latency

current position to a position adjacent to the read-write head. This latency has to be taken into account while calculating the overall latency.

Another example is the movement of a robotic arm; the time taken for the DC motor to move the rotor when the input changes, and also the time taken for the arm to move (out of inertia) to its final position constitutes a mechanical latency. Mechanical latency is a considerable part of the overall latency, in the case of multi-domain products.

Improving software performance

There are specific actions in software that can be considered in order to improve the performance of the product. Many of these come from empirical reasoning. All such performance-enhancing considerations are given below. Although many of these are design constraints, the architect needs to visualise these when designing.

- More floating-point operations used in the code will slow down the system.
- Use debug prints only if they are needed in the release load (the one given or released to the customer), since this will slow down the system. Debug prints could be liberally used in the debug load.
- Doing costly computations during the transaction will also slow down the system.
- Using many *sleep* calls might slow down the system and affect system performance. Instead, use semaphores for synchronisation. In general, do not use the *sleep* system call to maintain synchronisation; find out the exact cause of lack of synchronisation, and eliminate it.
- A number of short-duration transactions (such as a short-duration Internet connection, or a short call on your mobile network) have a tendency to eat up more CPU resources, and hence can slow down the system.
- The more the propagation delay through the stack, the lower the performance of the system. Propagation delay may be caused due to the layered architecture of device drivers, or the structure of the software itself.

- Creating more buffers during the transaction (when actual work is done in the product), and copying the data buffers more often, has a tendency to eat up system resources and hence can cause system slow-down.
- If the system is working in an IP network, more network traffic can have an effect on the system's performance.
- If we are implementing a layered device-driver architecture, using many call-back functions, this will affect system performance. Improper use of system APIs for communication will not help performance either. Layering in software improves maintainability and standardisation, but too much layering in code is not good for system performance.
- A monolithic driver implementation, as in Linux/UNIX, might help in improving system call performance—but maintainability will also decrease (see Reference 3).
- In this context, use of a proprietary operating system such as Cisco's IOS (Internetwork Operating System) improves system performance. The proprietary operating systems will have only the required features.
- Many switch/router/server manufacturing companies do have their own operating systems, which will yield a product that is easier to maintain, even though this will extend the time to market.
- A commercial operating system might have many overheads that would be overkill for our system under development, but there has to be a trade-off between more performance and maintainability, shorter gestation periods, or time to market. Finally, this is a decision taken not only by the designer, but also the product management and the strategy group head, in consultation with sales.
- System calls and heap allocations, if made in time-critical sections of code, have to be crisp and to the point.
- Code should be written with audits, so that memory leaks are checked often in the system. Audits will kick off in the system every 30 minutes (which is the norm), and will clean up unused file and semaphore handles, free unused memory areas, and do garbage collection, if needed. Doing all this periodically through audits is one of the important ways in which to keep a tab on the performance of a telecom product.
- Too many memory allocation failures should be avoided, since they can also slow down the system. This is because the system spends some CPU cycles to process the memory allocation request, and even though the request is denied, the system would have done some work.
- Locking and excessive interrupt handling should be done crisply in time-critical sections of code, since they affect system performance (see Reference 4).
- There is one fundamental reason why a system is slower: failure to use an optimal algorithm. Faulty algorithms might cause system degradation. For example, the wrong sorting algorithm in a specific place will result in a higher